

CHAPTER 3

Software Development Life Cycle & Quality Assurance

Coverage

10 Sections

Course

Software Engineering

Deck

52 Slides

Models

Waterfall·V-
Model·Agile·DevOps

Chapter 3 — Table of Contents

3.1 Introduction to SDLC

3.2 Waterfall Model and Quality Assurance

3.3 V-Model and Quality Assurance

3.4 Spiral Model and Quality Assurance

3.5 Agile Development and Quality Assurance

3.6 DevOps and Continuous Quality

3.7 Quality Assurance Activities Across SDLC Phases

3.8 Software Quality Planning

3.9 Quality Gates and Milestones

3.10 Chapter Summary

3.1

Introduction to SDLC

What is the Software Development Life Cycle?

Chapter 3 — SDLC and Quality Assurance

Section 3.1

What is the SDLC?

The Software Development Life Cycle (SDLC) is a structured process that defines the phases, activities, and deliverables for planning, creating, testing, and deploying an information system. It provides a framework for systematic development.

1

Planning

Define scope, feasibility, resources, schedule, cost estimates

2

Requirements

Gather, analyse, and document functional & non-functional requirements

3

Design

Architect the system: HLD (high-level) and LLD (low-level) design

4

Implementation

Write, review, and integrate source code according to design specs

5

Testing

Verify and validate software — unit, integration, system, UAT

6

Deployment & Maintenance

Release to production; monitor, fix, enhance over operational life

Section 3.1

Why SDLC Matters for Quality

The SDLC provides the scaffolding within which Quality Assurance activities are embedded:



Structured Framework

Provides a repeatable, systematic process for developing software — reduces ad-hoc decisions and inconsistency across projects.



Clear Milestones

Each phase has defined entry/exit criteria, deliverables, and quality checkpoints — making progress measurable and visible.



Risk Management

Early phase activities (planning, requirements) identify risks before they become costly failures in later phases.



Resource Planning

Structured phases allow accurate estimation of time, cost, and human resources needed at each development stage.



Quality Integration

SDLC phases provide natural integration points for QA activities: reviews, testing, audits, and quality gates.



Communication

Common SDLC framework creates shared vocabulary and expectations between developers, testers, managers, and clients.



Key principle: Quality cannot be tested in — it must be BUILT IN throughout the SDLC. QA starts at requirements, not testing.

Section 3.1

Overview of SDLC Models

Multiple SDLC models exist — each suited to different project types, sizes, and requirements stability:

Waterfall

197
0

Linear sequential phases. Each phase must complete before next begins. Best for stable, well-defined requirements.

✓ Simple, well-documented

✗ No iteration; late testing

V-Model

197
9

Extension of Waterfall with testing counterpart for each development phase. Strong verification & validation.

✓ Clear test planning

✗ Rigid; no iteration

Spiral

198
6

Risk-driven iterative model. Each spiral addresses risk analysis, planning, development, and evaluation.

✓ Risk management focus

✗ Complex; costly

Agile

200
1

Iterative, incremental development with frequent customer collaboration. Sprints deliver working software.

✓ Flexible; fast feedback

✗ Less predictable

DevOps

200
9

Extends Agile with continuous integration, delivery, and deployment. Breaks dev/ops silos.

✓ Continuous delivery

✗ Complex toolchain

Iterative/Incremental

197
5

Software built in small increments, each refined through iteration. Combines plan-driven and iterative elements.

✓ Early working software

✗ Needs good design upfront

3.2

Waterfall Model and Quality Assurance

The original sequential SDLC — Royce (1970).

Section 3.2

Waterfall Model — Phases & Characteristics

The Waterfall model (Winston W. Royce, 1970) is a linear sequential design process in which progress flows downward like a waterfall through phases:

1. Requirements Analysis

Collect and document all requirements. Output: SRS (Software Requirements Specification)

2. System Design

HLD + LLD. Architecture, DB design, interface design. Output: SDS (System Design Specification)

3. Implementation (Coding)

Actual coding based on design documents. Unit testing by developers. Output: Source code

4. Integration & Testing

System integration, functional testing, performance testing. Output: Test reports, bug reports

5. Deployment

Release to production environment. User training. Output: Deployed system, user manuals

6. Maintenance

Bug fixes, updates, enhancements. Ongoing support. Output: Updated releases, patches



Critical weakness: Testing happens late. A requirements defect found in testing may cost 40–100× more to fix than if caught during requirements review.

Section 3.2

QA Activities in Waterfall Model

QA must be planned for each phase — not just testing at the end:

Requirements

- Requirements review and walkthrough
- SRS inspection (IEEE 830)
- Testability analysis

📄 Reviewed SRS

System Design

- Design review and inspection
- Architecture quality assessment
- Interface design review

📄 Reviewed SDS

Implementation

- Code walkthroughs
- Code inspections (Fagan)
- Static code analysis tools

📄 Reviewed code

Testing

- Test plan review
- Test case review
- Integration test execution

📄 Test reports

Deployment

- Deployment checklist
- Release readiness review
- Production smoke testing

📄 Sign-off docs

Maintenance

- Change impact assessment
- Regression test execution
- Process metrics review

📄 Change reports

Section 3.2

Waterfall — Strengths, Weaknesses & Applicability

✓ Strengths

- Simple and easy to understand
- Clearly defined phases and milestones
- Strong documentation at each stage
- Easy to manage due to rigidity
- Well-understood by all stakeholders
- Good for fixed-price contracts

✗ Weaknesses

- No iteration — errors propagate forward
- Testing only at the end is risky
- Poor fit for changing requirements
- Customer sees product only at the end
- High risk for complex/uncertain projects
- Late defect discovery is very expensive

🎯 When to Use Waterfall:

- Requirements are clear, fixed, and well-documented
- Technology is well-understood and stable
- Project is short and team is experienced
- Regulatory environment requires extensive documentation

🚫 When NOT to Use:

- Requirements are unclear or likely to change
- Project is long (>6 months) with high uncertainty
- Customer needs early working software
- Complex, innovative, or exploratory projects

3.3

V-Model and Quality Assurance

Verification and Validation paired at every phase.

Section 3.3

The V-Model — Structure & Principles

The V-Model (Validation and Verification Model) extends Waterfall by aligning each development phase with a corresponding testing phase:

← *Verification (Building right?)*

Validation (Right product?) →

Requirements Analysis

Acceptance Testing (UAT)

System Design (HLD)

System Testing

Architecture Design

Integration Testing

Module Design (LLD)

Unit Testing

Coding

🔑 Key principle: For every development phase on the LEFT, there is a corresponding testing phase on the RIGHT. Test planning for the right side happens during the left side phase. Coding is at the bottom of the V.

Section 3.3

V-Model — Phase Correspondence Table

Each left-side phase generates test plans used in its right-side counterpart:

Development Phase	Test Level	What is Tested	Entry Criteria	Exit Criteria
Requirements Analysis	Acceptance Test	User requirements validated	Approved SRS	Signed SATP
System Design (HLD)	System Test	Complete system vs. specs	Approved SDS	System test report
Architecture Design	Integration Test	Component interactions	Design complete	Integration test report
Module Design (LLD)	Unit Test	Individual modules/functions	LLD complete	Unit test coverage
Coding	Code Reviews	Static analysis + peer review	Code complete	Review sign-off

✓ Strengths: Clear traceability · Test planning upfront · Strong V&V

✗ Weaknesses: Rigid · No iteration · Late integration risk

✓ V-Model advantage: Test planning begins at requirements phase — not when testing starts. This ensures comprehensive, traceable tests aligned with requirements.

3.4

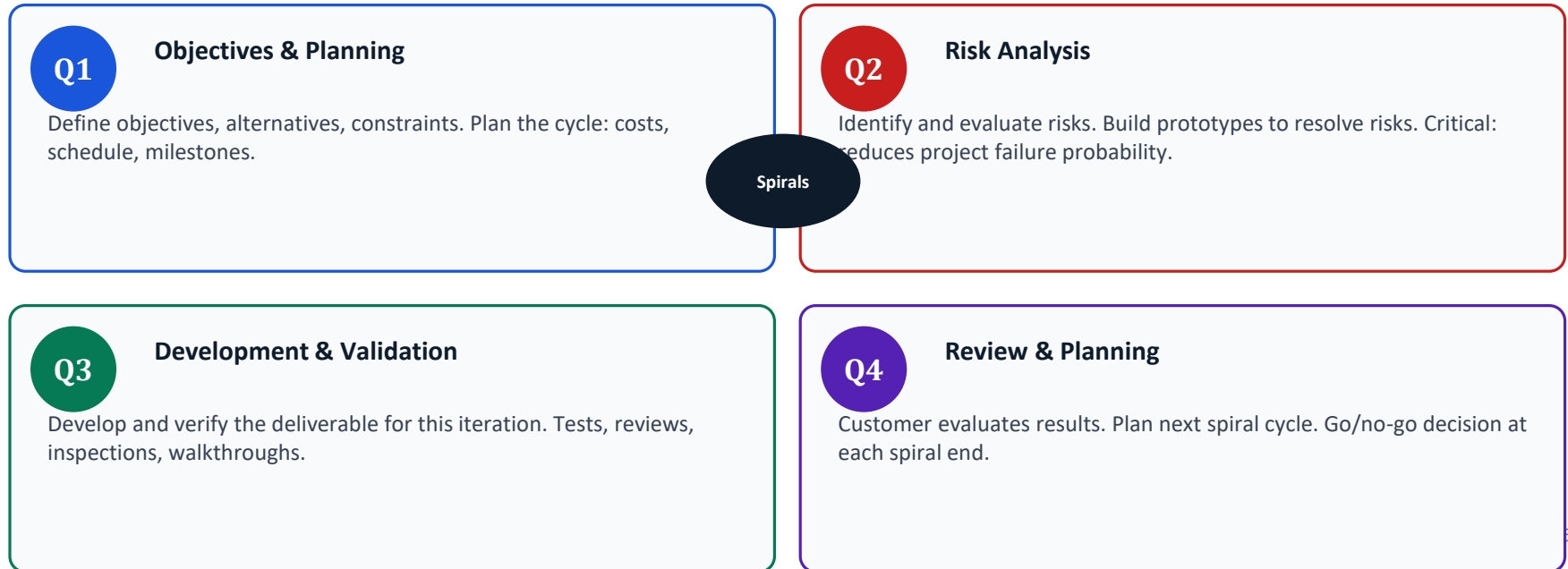
Spiral Model and Quality Assurance

Risk-driven iterative development — Boehm (1988).

Section 3.4

Spiral Model — Four Quadrants

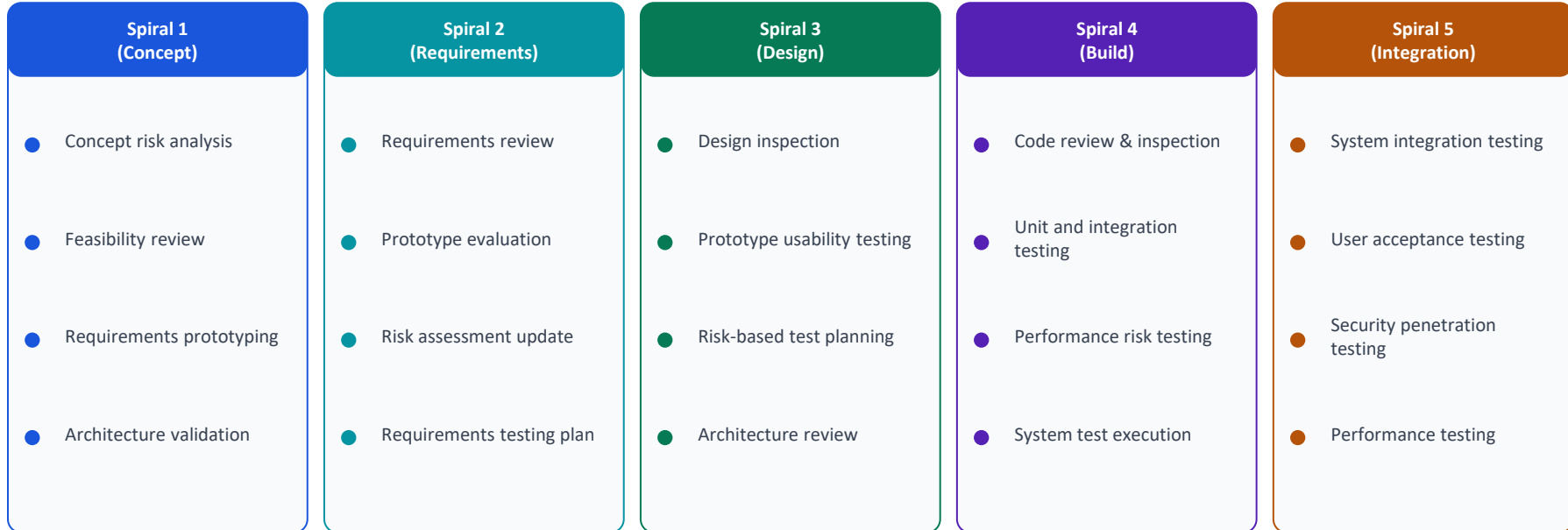
Barry Boehm's Spiral Model (1988) combines Waterfall and iterative development. Each spiral cycle passes through four quadrants:



Section 3.4

Spiral Model — QA & Risk Management

QA in the Spiral Model focuses especially on RISK IDENTIFICATION and RISK-DRIVEN TESTING:



✦ Spiral's greatest contribution to SQA: formal RISK ANALYSIS at each iteration prevents surprises. QA can prioritise testing based on

3.5

Agile Development and Quality Assurance

Quality built in through continuous collaboration.

Chapter 3 — SDLC and Quality Assurance

Section 3.5

Agile Principles & Quality

The Agile Manifesto (2001) prioritises: individuals & interactions, working software, customer collaboration, and responding to change. Quality is CONTINUOUS, not a phase:

Working Software over Documentation

🔍 QA Impact: QA verifies working features every sprint — not paper-based reviews

Customer Collaboration over Contract

🔍 QA Impact: Product Owner prioritises quality criteria; demos every sprint

Responding to Change

🔍 QA Impact: QA adapts test plans every sprint; regression suite grows incrementally

Individuals & Interactions

🔍 QA Impact: Whole team owns quality — not just testers. Devs write tests too

Section 3.5

Scrum Framework & QA Activities

Scrum is the most widely used Agile framework. QA activities occur throughout every Sprint:

Product Backlog Refinement

QA reviews user stories for testability. Adds acceptance criteria. Estimates testing effort.

Sprint Planning

QA agrees sprint test scope. Identifies regression risks. Plans test automation.

Sprint Execution (Daily Scrum)

Continuous testing alongside development. Daily sync on defects/blockers. TDD & ATDD.

Sprint Review (Demo)

QA validates acceptance criteria. Demonstrates test results. Customer feedback on quality.

Sprint Retrospective

Team reviews testing effectiveness. Improves QA process. Updates test automation strategy.

Definition of Done (DoD)

QA defines DoD: code reviewed, unit tests pass, integrated, acceptance criteria met, regression clean.

✦ In Scrum, quality is embedded in the Definition of Done (DoD) — software is not 'done' unless it meets all quality criteria defined by the team and QA.

Section 3.5

Agile QA Practices — TDD, BDD, ATDD

TDD — Test Driven Development

Red → Green → Refactor

Process:

- Write a FAILING test first (Red)
- Write MINIMUM code to pass the test (Green)
- REFACTOR code while tests stay green
- Repeat for each new behaviour

💡 Ensures 100% unit test coverage by design. Forces developers to think about API before implementation.

BDD — Behaviour Driven Development

Given → When → Then

Process:

- Write scenarios in plain English (Gherkin)
- Given [context] / When [action] / Then [outcome]
- Business + Dev + QA collaborate on scenarios
- Automated as acceptance tests (Cucumber/SpecFlow)

💡 Bridges communication between business and technical teams. Creates living documentation.

ATDD — Acceptance Test Driven Dev.

Define → Develop → Demo

Process:

- Define acceptance criteria BEFORE coding
- Acceptance tests written first (with QA + PO)
- Developers code until acceptance tests pass
- Tests become automated regression suite

💡 Ensures every feature meets customer requirements before development starts.

3.6

DevOps and Continuous Quality

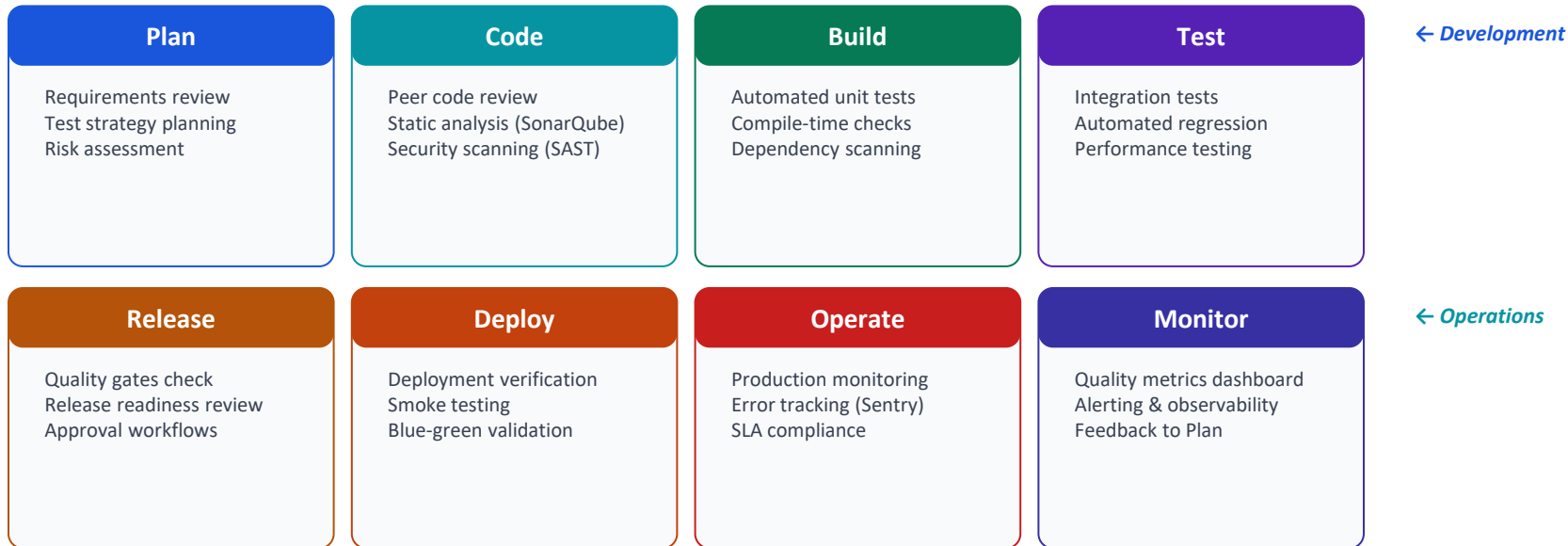
Breaking down silos — quality everywhere, always.

Chapter 3 — SDLC and Quality Assurance

Section 3.6

DevOps — Principles & Continuous Quality

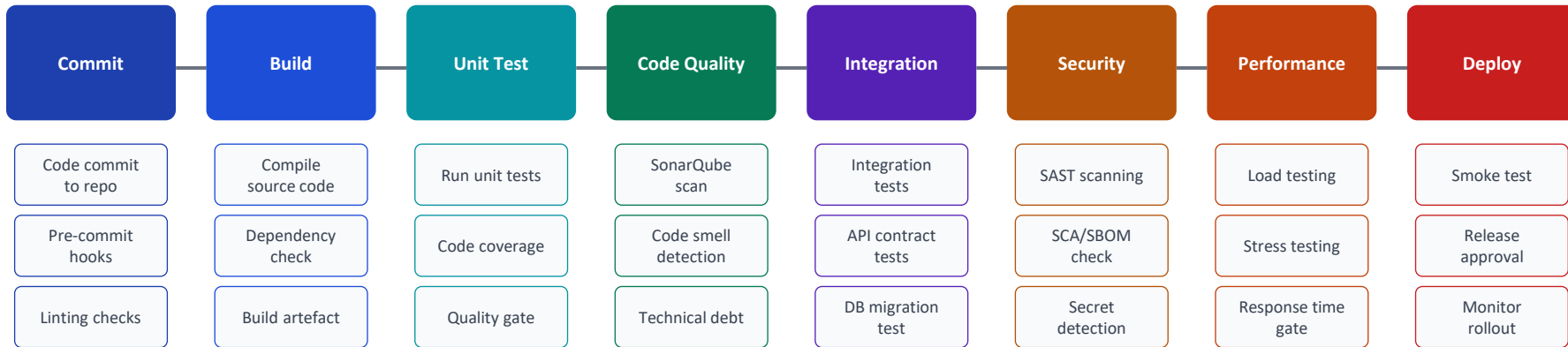
DevOps bridges Development and Operations, embedding quality assurance throughout the continuous delivery pipeline:



Section 3.6

CI/CD Pipeline — Quality Gates

A CI/CD (Continuous Integration/Continuous Delivery) pipeline automates quality checks at every stage:



Quality Gate = Automated Stop

If any gate fails → pipeline stops → team is notified → fix before proceeding.

- ✓ Benefits: Immediate defect feedback (minutes not days) · Consistent quality enforcement · Automated compliance checks · Auditability of every release.

Section 3.6

DevSecOps — Security in the Pipeline

DevSecOps integrates security quality assurance into every stage of the CI/CD pipeline:

SAST**Static Application Security Testing**

Analyses source code for security vulnerabilities without executing it. Finds SQL injection, XSS, buffer overflows at code level.

 *SonarQube, Checkmarx, Veracode, Bandit*

IAST**Interactive Application Security Testing**

Monitors application behaviour during testing. Combines SAST and DAST benefits. Very low false positive rate.

 *Contrast Security, Seeker*

**Secrets
Detecti
on****Hardcoded Credentials Scanning**

Scans code and configs for accidentally committed API keys, passwords, certificates, and tokens.

 *GitGuardian, TruffleHog, detect-secrets*

DAST**Dynamic Application Security Testing**

Tests running applications by simulating attacks. Finds runtime vulnerabilities not visible in source code.

 *OWASP ZAP, Burp Suite, Nikto*

SCA**Software Composition Analysis**

Identifies known vulnerabilities in third-party dependencies and open-source libraries (CVE database check).

 *Snyk, OWASP Dependency-Check, WhiteSource*

**Contain
er
Scannin
g****Image Vulnerability Assessment**

Scans Docker images and Kubernetes configs for vulnerabilities, misconfigurations, and compliance issues.

 *Trivy, Grype, Anchore, Clair*

3.7

QA Activities Across SDLC Phases

What QA does — phase by phase — in detail.

Chapter 3 — SDLC and Quality Assurance

Section 3.7

Phase-by-Phase QA Activities

Requirements

- Review SRS for completeness, consistency, clarity
- Verify testability of each requirement
- Create Requirements Traceability Matrix (RTM)
- Identify ambiguous or conflicting requirements
- Define acceptance criteria with stakeholders
- Begin test planning for system test

Design

- Conduct design reviews (peer review, walkthrough)
- Review database design for integrity constraints
- Review interface design for usability
- Assess architectural quality (modularity, coupling)
- Plan integration test strategy
- Review security design against threat model

Implementation

- Enforce coding standards (style guides)
- Conduct code walkthroughs and inspections
- Run static analysis tools (linting, SAST)
- Review unit tests written by developers
- Track and report defects found in code
- Monitor code coverage metrics

Section 3.7

Testing & Deployment QA Activities

Testing Phase

- Execute and manage the test plan
- Review and approve all test cases
- Conduct various test types: unit, integration, system, UAT
- Track, prioritise, and manage defects
- Perform regression testing after fixes
- Verify performance, security, usability

Deployment

- Define and check release readiness criteria
- Conduct deployment readiness review
- Execute post-deployment smoke tests
- Verify rollback procedures are in place
- User acceptance sign-off documentation
- Conduct final security assessment

Maintenance

- Impact analysis for every change request
- Regression test planning for changes
- Defect root cause analysis
- Configuration audit of deployed system
- Process metrics collection and analysis
- Lessons learned and improvement actions

Section 3.7

Testing Types Across the SDLC

Different test types are applied at different SDLC phases — forming the test pyramid:

Test Type	Who?	SDLC Phase	What Tested	Common Tools
Unit Testing	Developer / TDD	Coding	Individual modules, functions, classes	JUnit, NUnit, pytest, Jest
Integration Testing	Dev + QA	Post-coding	Component interactions, APIs, data flows	Postman, REST-assured, TestNG
System Testing	QA Team	After integration	Complete system vs. requirements	Selenium, TestComplete, LoadRunner
Performance Testing	Performance QA	Pre-release	Speed, load, stress, scalability	JMeter, Gatling, K6, Locust
Security Testing	Security QA	Pre-release	Vulnerabilities, penetration, OWASP	ZAP, Burp Suite, Metasploit
User Acceptance Test	Business users	Pre-deployment	Fitness for purpose, user workflows	Cucumber, FitNesse, Zephyr
Regression Testing	QA Team (auto)	After every change	Existing functionality not broken	Selenium, Cypress, Playwright
Smoke Testing	QA / DevOps	After deployment	Critical functions work in production	Automated scripts, health checks

3.8

Software Quality Planning

Planning quality into every stage of the project.

Chapter 3 — SDLC and Quality Assurance

Section 3.8

What is a Software Quality Plan (SQP)?

Software Quality Plan (SQP) / Software Quality Assurance Plan (SQAP)

A Software Quality Plan is a document that identifies the quality standards relevant to a project, describes the activities that will be conducted to achieve those standards, and defines the entry/exit criteria for each development phase. It is mandated by IEEE Std 730.

Key Components of a Software Quality Plan:

1

Project Purpose &**Scope:**

What software is being developed, for whom, and the SQP's coverage

3

Standards & Guidelines:

Coding standards, design standards, documentation standards to be followed

5

Test Plan:

Test approach, test types, test coverage, test entry/exit criteria, tools

7

Metrics &**Measurement:**

What will be measured, how, frequency, reporting, and thresholds

2

Quality Objectives:

Measurable quality goals (e.g., defect density < 0.5/KLOC, test coverage > 80%)

4

Reviews & Audits Plan:

Types, frequency, participants, and criteria for all reviews and audits

6

Defect Management:

How defects are reported, classified, tracked, and resolved

8

Tools & Techniques:

Tools for testing, static analysis, CM, defect tracking, reporting

Section 3.8

Quality Planning Process

Quality planning follows a systematic process aligned to the project lifecycle:

Step 1	Understand Requirements:	Review project requirements, constraints, customer quality expectations, applicable standards (ISO, IEEE), and regulatory requirements.
Step 2	Define Quality Objectives:	Set SMART quality objectives: Specific, Measurable, Achievable, Relevant, Time-bound. E.g., 'Zero critical defects at UAT sign-off'.
Step 3	Identify Standards & Regulations:	Identify applicable coding, testing, documentation, and security standards. Map regulatory requirements to QA activities.
Step 4	Plan QA Activities:	Define what reviews, inspections, audits, and tests will be conducted. Assign responsibilities. Schedule activities in project plan.
Step 5	Define Metrics & Thresholds:	Specify quality metrics to track (defect density, coverage, MTBF). Define thresholds and acceptable ranges for each metric.
Step 6	Define Quality Gates:	Specify entry and exit criteria for each phase. What quality evidence is required to move to the next phase?
Step 7	Plan Resources & Tools:	Identify SQA team, skills needed, tools required (test management, automation, static analysis), and budget for quality activities.
Step 8	Review & Approve Plan:	Submit SQAP for review and approval by project manager, quality manager, and customer. Baseline the plan.

Section 3.8

Quality Metrics in Planning

Quality metrics must be defined in the quality plan — before development begins:

Process Metrics

Measure the software development process itself

- Defect removal efficiency (DRE) — % defects removed before release
- Review coverage — % of work products reviewed
- Inspection defect density — defects found per review hour
- Requirements stability — % of requirements changed

Product Metrics

Measure characteristics of the software product

- Defect density — defects per KLOC
- Code coverage — % of code exercised by tests
- Cyclomatic complexity — code complexity measure
- MTBF — Mean Time Between Failures in test

Project Metrics

Measure project progress and performance

- Test execution progress — % test cases run/passed
- Open defect age — average age of unresolved defects
- SQA effort ratio — % project effort on SQA
- Schedule adherence — QA milestones on time

3.9

Quality Gates and Milestones

Checkpoints that enforce quality before progression.

Section 3.9

What are Quality Gates?

Quality Gate Definition

A Quality Gate is a mandatory checkpoint between SDLC phases that defines the minimum quality criteria that must be met before the project can progress to the next phase. Quality gates are BINARY — either PASSED (proceed) or FAILED (do not proceed).

Quality Gates by Phase:

Requirements → Design

- SRS reviewed and approved by all stakeholders
- All requirements are testable (measurable acceptance criteria)

+ 2 more criteria

Implementation → Testing

- All code reviewed by peers
- Unit test coverage ≥ 80% (or project standard)

+ 2 more criteria

Design → Implementation

- Design reviewed and approved (PDR/CDR passed)
- All interfaces defined and agreed

+ 2 more criteria

Testing → Deployment

- All planned test cases executed
- Zero open CRITICAL or HIGH severity defects

+ 4 more criteria

Section 3.9

Entry & Exit Criteria for Each Phase

Entry criteria must be met BEFORE entering a phase. Exit criteria must be satisfied BEFORE leaving:

Phase	Entry Criteria (Must have before starting)	Exit Criteria (Must achieve before proceeding)
Requirements	• Approved project charter • Stakeholders identified • Feasibility study complete	• Approved SRS • RTM created • All requirements testable
Design	• Approved SRS • Technical team assigned • Tools/env. available	• Approved SDS (HLD + LLD) • Design reviewed • Test plans started
Implementation	• Approved SDS • Development environment ready • Coding standards defined	• All modules coded • Unit tests $\geq$ 80% coverage • Code reviewed, defects fixed
Testing	• Code review complete • Test environment set up • Test data prepared	• All test cases run • Zero critical defects open • Regression suite green
Deployment	• UAT sign-off • Rollback plan documented • Deployment procedures approved	• Smoke tests pass • Monitoring enabled • Release notes complete
Maintenance	• Deployed system stable • Support team trained • Monitoring active	• All SLAs met • Change process established • Documentation updated

Section 3.9

SonarQube Quality Gates — Example

SonarQube is the most widely used automated quality gate tool. Here is a typical SonarQube quality gate configuration:

Quality Metric	Operator	Threshold	Gate	Impact
Coverage on New Code	is less than	80%	FAIL	Pipeline blocked
Duplicated Lines on New Code	is greater than	3%	FAIL	Pipeline blocked
Maintainability Rating	is worse than	A	FAIL	Pipeline blocked
Reliability Rating	is worse than	A	FAIL	Pipeline blocked
Security Rating	is worse than	A	FAIL	Pipeline blocked
Security Hotspots Reviewed	is less than	100%	FAIL	Pipeline blocked
Blocker Issues	is greater than	0	FAIL	Pipeline blocked
Critical Issues	is greater than	0	FAIL	Pipeline blocked

☒ When ALL conditions pass: Gate = PASSED (green) → Pipeline continues. If ANY condition fails: Gate = FAILED (red) → Pipeline stops. Fix

3.10

Chapter Summary

Key takeaways from Chapter 3.

Chapter 3 — SDLC and Quality Assurance

Section 3.10

Chapter 3 — Key Concepts Summary

● SDLC Definition:	Structured process defining phases, activities, and deliverables for planning, creating, testing, and deploying software systems.
● Waterfall (1970):	Linear, sequential phases. Each phase fully complete before next. Strong documentation but late testing. Best for stable, fixed requirements.
● V-Model (1979):	Each development phase paired with a testing phase. Test plans created during development. Strong verification & validation — but still rigid.
● Spiral (1988):	Risk-driven iterative model. Four quadrants per cycle: Objectives, Risk Analysis, Development, Review. Formal risk management at each iteration.
● Agile (2001):	Iterative sprints, customer collaboration, working software over documentation. TDD, BDD, ATDD. Definition of Done (DoD) embeds quality.
● DevOps:	Continuous integration/delivery. Quality gates in CI/CD pipeline. Shift-left (test early) + shift-right (test in production). DevSecOps adds security.
● QA Across SDLC:	Requirements: RTM, testability review. Design: inspections. Code: static analysis, reviews. Test: all test types. Deploy: smoke test, UAT sign-off.
● Quality Planning:	SQAP defines quality objectives, standards, activities, metrics, and quality gates. IEEE 730 mandates the structure. 8-step quality planning process.
● Quality Gates:	Binary checkpoints: PASS or FAIL. Entry criteria (before) + Exit criteria (after). SonarQube automates code quality gates in CI/CD pipelines.
● Test Types:	Unit → Integration → System → Performance → Security → UAT → Regression. Each type applied at appropriate SDLC phase.
● CI/CD Quality:	Automated pipeline: commit → build → unit test → static analysis → integration → security scan → performance → deploy with quality gates at each stage.
● SQA Principle:	Quality cannot be tested in — it must be built in throughout. SQA is proactive (prevents defects) not reactive (finds defects after creation).

SDLC Models — Comprehensive Comparison

Aspect	Waterfall	V-Model	Spiral	Agile	DevOps
Year	1970	1979	1988	2001	2009+
Structure	Linear	V-shaped	Spiral	Iterative	Continuous
Iteration	None	None	Per spiral	Per sprint	Continuous
Risk handling	Low	Low	High (explicit)	Medium	High
Customer involvement	Start & end	Start & end	Each cycle	Every sprint	Continuous
Documentation	Extensive	Extensive	Moderate	Minimal	Automated
Testing timing	End only	Each phase	Each spiral	Every sprint	Continuous CI
Flexibility	Very rigid	Rigid	Moderate	Very flexible	Highly flexible
Team size	Any	Medium-large	Large	Small	Cross-functional
QA integration	Phase-end	Per phase	Per spiral	Built-in	Pipeline gates
Best for	Stable fixed req.	Safety-critical	High-risk projects	Changing req.	Continuous delivery
Not good for	Changing req.	Flexible projects	Small/simple	Compliance-heavy	Legacy systems

Key Terms & Definitions

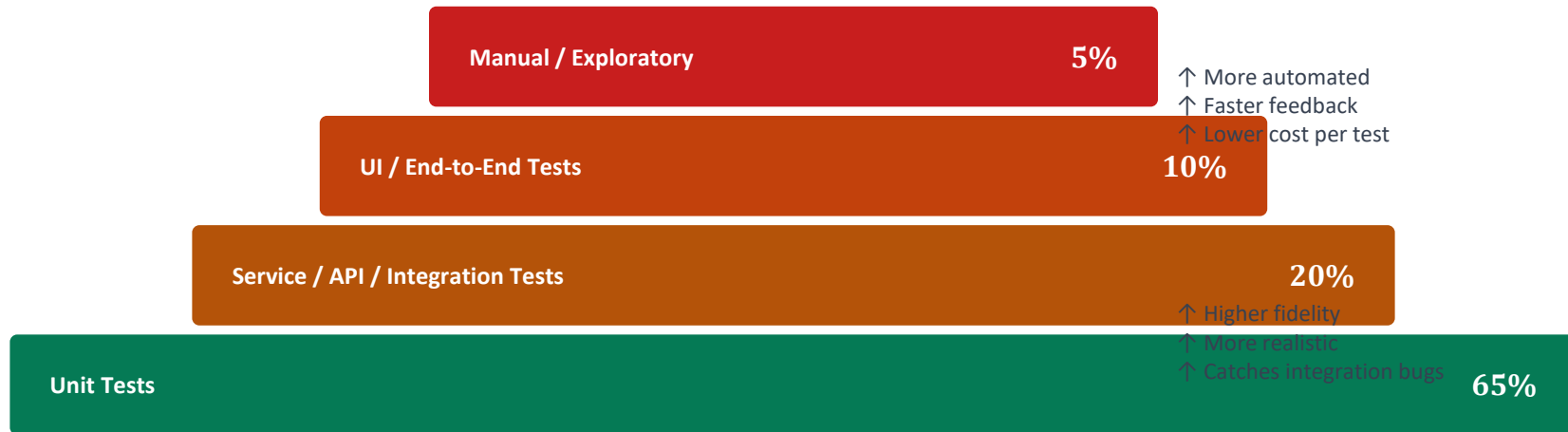
SDLC	Software Development Life Cycle — structured process for planning, creating, testing, and deploying software.
Waterfall	Linear sequential SDLC model where each phase must be fully completed before the next begins.
V-Model	SDLC model pairing each development phase with a testing counterpart. Strong verification and validation.
Spiral Model	Risk-driven iterative SDLC model by Boehm. Four quadrants: objectives, risk analysis, development, review.
Agile	Iterative, incremental SDLC approach emphasising collaboration, working software, and adaptability to change.
Scrum	Most popular Agile framework using time-boxed sprints, daily scrums, sprint reviews, and retrospectives.
DevOps	Cultural and technical movement combining Dev and Ops for continuous integration, delivery, and deployment.
CI/CD	Continuous Integration/Continuous Delivery — automated pipeline for building, testing, and deploying software.
TDD	Test Driven Development — write failing test FIRST, then write minimum code to pass, then refactor.
BDD	Behaviour Driven Development — define behaviour in Given/When/Then scenarios before coding.
ATDD	Acceptance Test Driven Development — acceptance criteria defined before development begins.
Quality Gate	Binary checkpoint requiring defined quality criteria to be met before proceeding to the next phase.
DoD	Definition of Done — explicit checklist of quality criteria that must be satisfied before a story is 'done'.
RTM	Requirements Traceability Matrix — maps requirements to test cases ensuring full test coverage.
SQAP	Software Quality Assurance Plan — document defining QA activities, standards, metrics, and responsibilities.
Shift-Left	Testing earlier in the SDLC to detect defects when they are cheapest to fix.
Shift-Right	Testing in production environments using monitoring, canary releases, chaos engineering.

Review & Practice Questions

-
- Q1** Define the Software Development Life Cycle (SDLC). Why is it important for Software Quality Assurance?
- Q2** Describe the Waterfall Model. What are its six phases? What are its main advantages and disadvantages from a QA perspective?
- Q3** Explain the V-Model. How does it differ from Waterfall? Create a table showing the correspondence between development and testing phases.
- Q4** Describe the Spiral Model's four quadrants. How does its risk-analysis approach benefit software quality?
- Q5** What is Agile development? How does it incorporate quality assurance differently from traditional models?
- Q6** Explain Test Driven Development (TDD). What are the three steps (Red-Green-Refactor)? What benefits does it provide?
- Q7** What is DevOps? Describe a typical CI/CD pipeline and explain how quality gates are implemented at each stage.
- Q8** What is a Software Quality Plan (SQAP)? List and briefly describe its key components.
- Q9** Define Quality Gates. What is the difference between entry criteria and exit criteria? Give examples for two SDLC phases.
- Q10** Compare Agile QA practices with traditional (Waterfall) QA. Create a table showing at least 8 differences.
- Q11** What is DevSecOps? Describe four security testing tools used in a DevSecOps pipeline.
- Q12** Explain the concept of 'shift-left testing'. How does it reduce the cost of defects?

The Test Pyramid — Strategy for Test Automation

Mike Cohn's Test Pyramid (2009) guides the proportion of different test types:



🚫 Anti-pattern: 'Ice cream cone' — more manual/UI tests than unit tests. This is SLOW, EXPENSIVE, and FRAGILE. Always build your pyramid right-side up with unit tests as the foundation.

Chapter 3 Quick Reference Card — SDLC & QA

Print this slide and keep it at your desk

Waterfall

Linear sequential
6 phases, no iteration
Testing at end only
Best: stable requirements
Key doc: SRS → SDS → STP
Risk: late defect detection

V-Model

Dev + Test mirrored
Each phase has test counterpart
Test plans written early
Best: safety-critical systems
Entry/exit criteria per phase
Strong V&V traceability

Spiral

Risk-driven iteration
4 quadrants per cycle
Prototype to resolve risks
Best: high-risk projects
Formal risk management
Costly but thorough

Agile/Scrum

2-4 week sprints
DoD enforces quality
TDD/BDD/ATDD
Whole team owns quality
Backlog → Plan → Demo → Retro
Best: changing requirements

DevOps CI/CD

Continuous pipeline
Quality gates at each step
Shift-left + shift-right
SonarQube code gates
DevSecOps: SAST/DAST
Monitor in production

Quality Planning

SQAP (IEEE 730)
Quality objectives
Entry/exit criteria
Quality gates = PASS/FAIL
Metrics: DRE, coverage
Review, audit, test plan

Waterfall vs. Agile — QA Comparison

QA Aspect	Waterfall	Agile
QA timing	End of development cycle	Continuous — every sprint
Test planning	Upfront, comprehensive document	Rolling — per sprint
Customer involvement	At UAT only (end)	Every sprint review/demo
Defect feedback cycle	Weeks or months	Hours or days
Regression testing	Planned, periodic	Automated, every build
Documentation	Extensive, formal	Lightweight living docs
Test automation	Optional, limited	Essential for speed
Team responsibility	QA team owns testing	Whole team owns quality
Requirements changes	Formal change control	Welcomed, planned in sprint
Risk management	Phase-gate reviews	Sprint retrospectives + risk board
Quality definition	Conformance to SRS	Definition of Done (DoD)

Software Quality Assurance Plan (SQAP) Template

IEEE 730 compliant SQAP structure for a typical software project:

1.0

Purpose & Scope

Project overview · Software product description · Applicability of this plan · Document structure

3.0

Management

Organisation chart · QA responsibilities matrix · RACI chart · Tools & resources · Schedule of QA activities

5.0

Standards & Conventions

Coding standards · Naming conventions · Design methodologies · Test case format · Review checklists

7.0

Test Plan

Test strategy · Test levels · Test types · Entry/exit criteria · Test tools · Test data management

9.0

Tools & Techniques

Static analysis tools · Test management tools · Automation frameworks · Defect tracking · Version control

2.0

Reference Documents

IEEE 730 · Project charter · SRS · SDS · Coding standards · Testing standards · Regulatory standards

4.0

Documentation Plan

Required documents: SRS, SDD, STP, STR, SCM Plan · Review/approval process · Document control

6.0

Reviews & Audits

Requirements review · Design review · Code inspection · Process audit · Product audit · Post-project audit

8.0

Problem Reporting

Defect report format · Severity classification · Priority classification · Resolution process · Metrics & reporting

10.0

Metrics & Improvement

Quality metrics defined · Measurement frequency · Threshold values · Reporting to management · Improvement actions

Agile vs. DevOps — Relationship & Differences

Agile and DevOps are complementary — DevOps extends Agile to include Operations and continuous delivery:

Agile Only	Both	DevOps Only
● Sprint-based delivery ● User story driven ● Product Backlog ● Scrum ceremonies ● Story points ● Burn-down charts ● Sprint velocity	● Iterative delivery ● Customer focus ● Continuous improvement ● Automated testing ● Cross-functional teams ● Fast feedback loops ● Quality first	● CI/CD pipelines ● Infrastructure as Code ● Containerisation (Docker) ● Production monitoring ● Blue-green deployments ● Feature flags ● SRE practices

Exam Tips — Chapter 3

Key areas to focus on for examinations:

SDLC Models (Must Know)

- All 5 models: year, author, key characteristics
- Waterfall phases (6) and their sequence
- V-Model: which dev phase maps to which test phase
- Spiral: 4 quadrant names and what happens in each
- Agile: sprint cycle, Scrum events, DoD

Common Exam Questions

- Draw and explain the V-Model diagram
- Compare Waterfall and Agile QA (table)
- Describe the CI/CD pipeline with quality gates
- Explain quality gates — entry vs. exit criteria
- Define TDD with Red-Green-Refactor cycle

QA per Model (Must Know)

- Waterfall: when testing occurs and why it's a weakness
- V-Model: test plans written during dev phases
- Agile: TDD/BDD/ATDD explained with examples
- DevOps: CI/CD pipeline stages and quality gates
- Quality gate = binary PASS or FAIL checkpoint

Calculations & Application

- Cost of fixing defects: $\text{requirements} \times 1$ vs $\text{production} \times 100$
- Test pyramid proportions: unit (65%), integration (20%), UI (10%)
- Defect density = $\text{defects} \div \text{KLOC}$
- DRE = $(\text{defects found pre-release}) \div (\text{total defects}) \times 100\%$

References & Further Reading

- **Royce, W.W. (1970)**
"Managing the Development of Large Software Systems." Proc. IEEE WESCON, pp. 328–338.
- **Boehm, B.W. (1988)**
"A Spiral Model of Software Development and Enhancement." IEEE Computer, 21(5), pp. 61–72.
- **Beck, K. et al. (2001)**
Manifesto for Agile Software Development. agilemanifesto.org
- **Cohn, M. (2009)**
Succeeding with Agile: Software Development Using Scrum. Addison-Wesley.
- **Humble, J. & Farley, D. (2010)**
Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.
- **Pressman, R.S. & Maxim, B.R. (2019)**
Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
- **IEEE Std 730 (2014)**
IEEE Standard for Software Quality Assurance Processes. IEEE Computer Society.
- **Kim, G., Debois, P., Willis, J. & Humble, J. (2016)**
The DevOps Handbook. IT Revolution Press.
- **Meszaros, G. (2007)**
xUnit Test Patterns: Refactoring Test Code. Addison-Wesley.
- **Chrissis, M.B., Konrad, M. & Shrum, S. (2011)**
CMMI for Development, Version 1.3. Addison-Wesley.

Practice Exercises — Chapter 3

Exercise 1

SDLC Model Selection

You are the QA Lead for three different projects: (a) Military radar system with fixed government requirements, (b) E-commerce mobile app with evolving features, (c) New machine learning product — unknown requirements. Recommend an SDLC model for each and justify your choice.

Exercise 2

V-Model Mapping

Given these requirements: 'User must be able to log in with email and password. Login must respond in < 2 seconds. Invalid login must show error message.' Create a V-Model test plan showing which test level addresses each requirement.

Exercise 3

Quality Gate Design

Design a complete set of quality gates for a 6-month web application project. For each gate: define what phase it sits between, list 5 entry criteria, list 5 exit criteria, and describe what happens when the gate fails.

Exercise 4

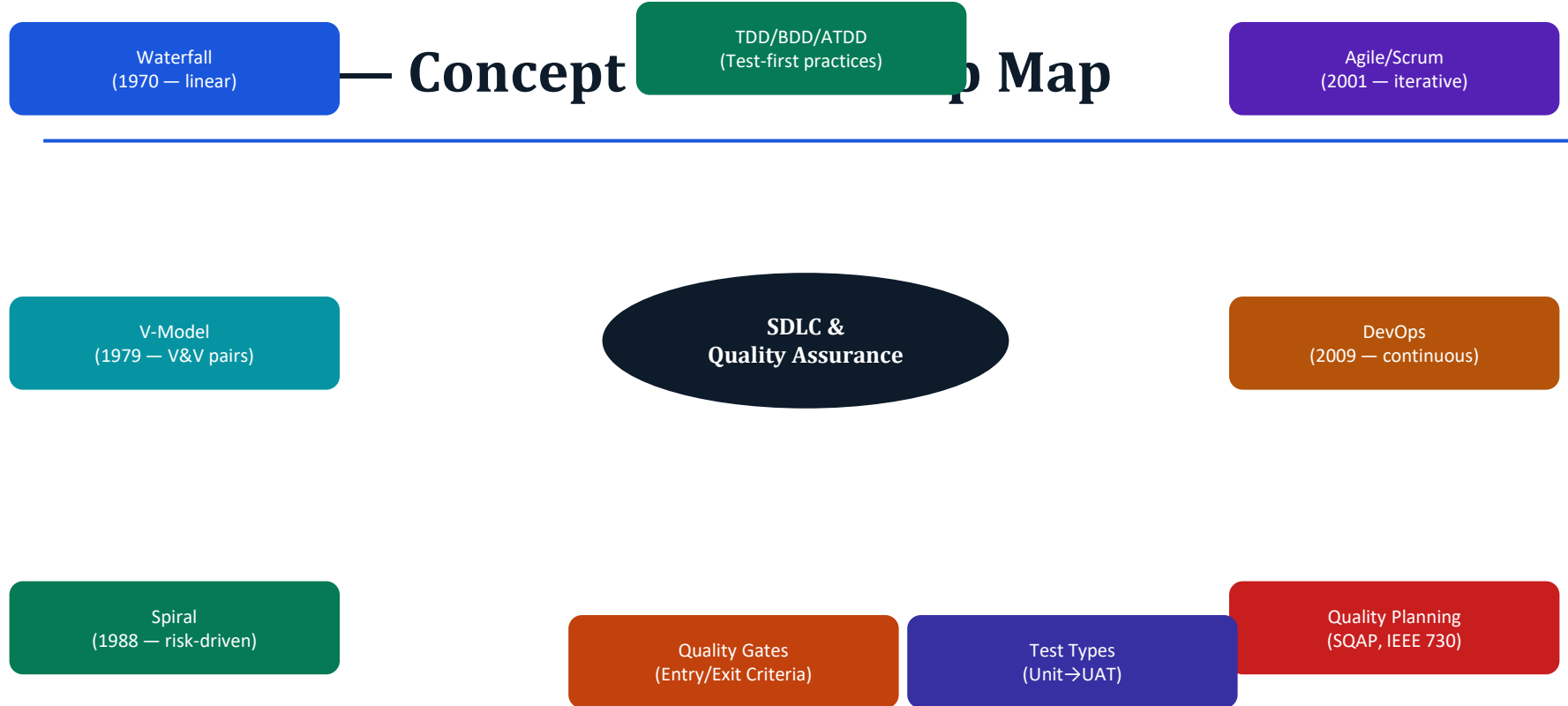
TDD Implementation

Apply TDD to implement a 'Password Validator' function that: (1) must be 8+ chars, (2) must contain a number, (3) must contain a special character. Write the failing tests first, then the minimum code to pass them, then refactor.

Exercise 5

CI/CD Pipeline Design

Design a CI/CD pipeline for a Python web application. Include: all stages, the quality check at each stage, the tool used, what passes/fails the gate. Justify why each stage is necessary for quality assurance.



Test Management Across SDLC Models

How test management activities differ across SDLC models:

Activity	Waterfall	V-Model	Agile	DevOps
Test Planning	Upfront, complete	Per phase, structured	Per sprint, adaptive	Automated in pipeline
Test Design	After requirements	Alongside dev phases	Sprint-by-sprint	Auto-generated + manual
Test Execution	Test phase only	Each test level	Daily in sprint	Continuous automation
Defect Management	Formal bug tracker	Formal, phase-based	Sprint backlog	Auto-alert in pipeline
Regression Testing	Planned, periodic	Per phase completion	Automated, per sprint	Every commit/build
Test Reporting	End-of-phase reports	Test level reports	Sprint demo + burndown	Real-time dashboards
Test Tools	Manual + scripts	Test frameworks	TDD frameworks + Jira	Selenium, JMeter, Sonar
Traceability	RTM document	RTM per phase	User story acceptance criteria	Pipeline test results
Coverage tracking	Test phase metrics	Test level metrics	Story coverage	Code coverage in pipeline
Test environment	Dedicated test env	Per-phase test env	Sprint test environment	Ephemeral, auto-provisioned
Release decision	Manual sign-off	Phase-gate approval	Sprint review demo	Automated quality gate
Continuous improvement	Post-project review	Post-phase review	Sprint retrospective	Pipeline metrics dashboard